

PACE: Stage-Consistent Long-Horizon Robot Manipulation via Progress-Aligned Context for Execution

Yenan Chen^{1,†}, Junjie Shi¹, Lu Chen¹, Zhongxiang Zhou¹, and Rong Xiong^{1,*}

Abstract—Demonstration-conditioned policies provide a natural interface for specifying robot behavior, yet long-horizon manipulation remains difficult when visually similar states recur across different stages or when demonstrations and executions proceed at different speeds. We identify the resulting failure mode as *stage confusion* and introduce Progress-Aligned Context for Execution (PACE), a stateful method that continually reinterprets a complete demonstration according to realized execution progress. PACE compresses the demonstration into ordered multimodal prompt tokens and uses training-only dual-edge attention supervision to expose its latent stage structure. During execution, an episode-local fast-weight memory causally encodes realized action–observation transitions and modulates prompt cross-attention, producing a progress-aligned context for a unified diffusion action expert without test-time stage labels or stage-specific policies. PACE improves success from 88.9% to 94.0% on LIBERO-Gen Goal Chain, from 79.1% to 83.3% on Spatial Combination, and from 33.3% to 73.3% on the two-step Block Routing tasks. Failure analysis further indicates that structured demonstration alignment and causal execution memory jointly mitigate stage confusion.

I. INTRODUCTION

Enabling robots to adapt to new tasks from minimal user input without retraining is a central objective of general-purpose robotic manipulation. Recent visuomotor policies have increasingly supported task conditioning through language, goal images, and multimodal demonstrations [1]–[4]. Among these, a perception–action demonstration conveys the global task objective, the sequence of operations, and fine-grained motion details, enabling policies to adapt to new execution objectives without updating model parameters [2], [4]–[9].

However, long-horizon multi-stage manipulation requires a policy to continually track task progress and interpret the demonstration relative to the current execution state. This is particularly challenging when different stages contain similar or recurring observations: similar states may require different actions depending on what has already been completed. Retrieving a demonstration segment inconsistent with the current progress may cause the policy to regress to an earlier stage, skip or repeat a stage, or terminate prematurely. We refer to this class of failures as **stage confusion**, illustrated in Fig. 1. Stage confusion primarily arises from two related factors: **cross-stage observation aliasing**, whereby distinct latent stages exhibit similar observations, and **demonstration–execution temporal-scale mismatch**, whereby the robot

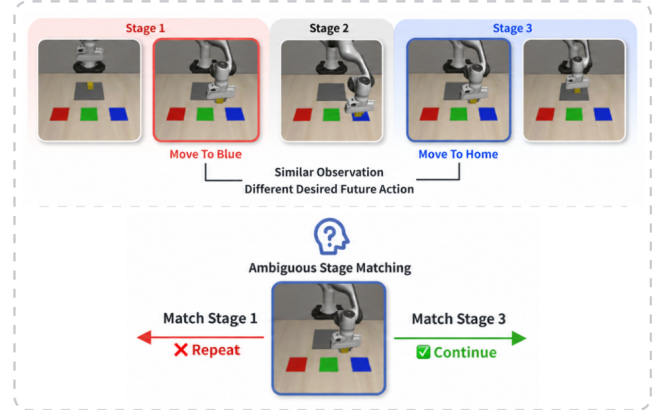


Fig. 1. Stage confusion in long-horizon demonstration-conditioned manipulation. Visually similar observations can recur at different stages while requiring different future actions (top). Observation-only matching may therefore retrieve an earlier stage and repeat completed behavior instead of continuing from the current progress (bottom).

and the demonstrator complete the same behavioral stage at different speeds.

Existing approaches address this problem only partially. Explicit stage or subgoal models often require predefined task decompositions, semantic subgoals, or manual stage annotations [10]–[13], while frame- or window-level localization is vulnerable to recurring observations and execution-speed variation [14], [15]. Long-context and memory-augmented policies retain longer execution histories [16]–[19], but typically use history as additional input to action prediction rather than to continually revise which part of the demonstration is relevant. These limitations motivate a mechanism that recovers stage structure from demonstrations without manual annotations and uses causal execution history to dynamically determine which part of the demonstration should guide the current behavior.

Event cognition research suggests that humans parse continuous activity into structured events and update an active event model at perceptual boundaries [20]–[22]. Inspired by this view, we adopt a functional abstraction in which an event model is realized not necessarily through parametrically independent modules, but through context dynamically constructed during execution. Specifically, the policy combines current perception with within-episode execution history to locate and aggregate demonstration segments consistent with the current progress, forming an evolving event context that enables a shared policy to produce stage-appropriate behavior.

¹All authors are with the College of Control Science and Engineering, Zhejiang University, Hangzhou 310027, China.

[†]Contact: Yanan Chen (12532108@zju.edu.cn).

^{*}Corresponding author: Rong Xiong (rxiong@zju.edu.cn).

Building on this insight, we introduce **Progress-Aligned Context for Execution (PACE)**, a stateful progress-alignment method for long-horizon demonstration-conditioned manipulation. During training, PACE automatically extracts and monotonically aligns action-stage boundaries, imposing dual-edge attention supervision and effective-support regularization. During execution, an episode-local fast-weight memory causally encodes realized action–observation transitions and modulates observation queries before cross-attention with the full demonstration. The resulting progress-aligned demonstration context conditions a unified diffusion action expert. Stage boundaries are used only for training supervision; at inference time, PACE requires neither explicit event segmentation, discrete stage prediction, nor stage-specific policies.

We evaluate PACE on compositional long-horizon manipulation benchmarks and real-world robotic tasks. On the LIBERO-Gen Spatial Combination and Goal Chain, PACE improves the success rate from 79.1% to 83.3% and from 88.9% to 94.0%. On the two-step Block Routing tasks, which exhibit pronounced cross-stage observation aliasing, PACE raises the mean success rate from 33.3% to 73.3%. Stage-level analyses further show that structured demonstration alignment and causal execution memory work synergistically to suppress stage confusion.

Our main contributions are as follows:

- We systematically study **stage confusion** in long-horizon demonstration-conditioned manipulation, attributing it to cross-stage observation aliasing and demonstration–execution temporal-scale mismatch.
- We introduce **PACE**, centered on a causally updated, episode-local fast-weight state that encodes realized action–observation transitions and modulates observation queries before prompt cross-attention, allowing visually similar current observations to attend to different demonstration stages under different execution histories. PACE further combines automatic stage-structure discovery with dual-edge attention supervision and effective-support regularization, without requiring manual stage annotations or stage-specific policies.
- We validate PACE on compositional simulation tasks and real-world robotic tasks with held-out demonstrations, and use stage-level analyses to elucidate how it mitigates stage confusion.

II. RELATED WORK

A. Demonstration-Conditioned Robot Policies

Complete perception–action demonstrations provide a natural inference-time interface for specifying task objectives, operation sequences, and motion details without updating policy parameters. Existing methods relate human demonstrations to robot observations through cross-attention or learned cross-embodiment representations [2], [8], retrieve or align relevant demonstration segments [23], [24], encode task identity from one or a few demonstrations [5], [6], or jointly model demonstration and execution trajectories as prompted,

in-context sequences [3], [4], [9]. In particular, Behavior Prompting Policy (BPP) treats a complete perception–action demonstration as a behavioral prompt for closed-loop action generation [4]. However, these approaches generally use the demonstration as a static conditioning sequence and do not continually reinterpret it according to realized execution progress, leaving them susceptible to cross-stage observation aliasing.

B. Temporal Alignment and Stage-Structured Manipulation

Temporal correspondence methods, including frame-level metric learning, temporal cycle consistency, dynamic time warping, and optimal transport, align sequences with different appearances or execution speeds [14], [15], [25], [26]. Local correspondences become ambiguous when visually similar states recur, while sequence-level alignments are typically computed offline or remain fixed during execution. Related hierarchical approaches represent progress using skills, options, subgoals, or keyframes [10]–[13], but often require predefined decompositions, semantic labels, or separate skill policies. Although unsupervised skill discovery reduces annotation requirements [27], it does not directly establish an execution-conditioned correspondence with a new demonstration. Latent-plan methods learn reusable long-horizon structure from play or human video [28], [29], but likewise do not continually realign a provided demonstration using the realized execution history. PACE instead learns ordered demonstration structure without manual stage labels and dynamically interprets that structure during execution.

C. History and Memory for Long-Horizon Manipulation

Long-horizon manipulation is partially observable because similar current observations may require different actions depending on prior interactions. Recurrent policies, history-aware Transformers, and memory-augmented models address this problem by retaining execution history [16], [17]. Test-time training further represents history as recurrently updated fast weights [18], and RoboTTT applies this mechanism to long-horizon visuomotor control [19]. These approaches primarily use history to improve action prediction. In contrast, PACE uses an episode-local fast-weight state to modulate observation queries before prompt cross-attention, allowing execution history to determine which part of the demonstration guides the current action.

III. PROBLEM STATEMENT

We consider long-horizon manipulation conditioned on a single perception–action demonstration $\mathcal{P} = \{(o_j^p, a_j^p)\}_{j=1}^{T_p}$. At execution timestep t , the policy predicts an action chunk of horizon H from the current observation and strictly causal episode history:

$$\mathcal{H}_t = \{(o_k^e, a_k^e, o_{k+1}^e)\}_{k=1}^{t-1}, \quad (1)$$

$$a_{t:t+H-1}^e \sim \pi_\theta(\cdot \mid o_t^e, \mathcal{H}_t, \mathcal{P}).$$

Here, o_j^p and a_j^p respectively denote the observation and action at demonstration timestep j , and T_p is the demonstration length. Each tuple $(o_k^e, a_k^e, o_{k+1}^e)$ denotes a realized

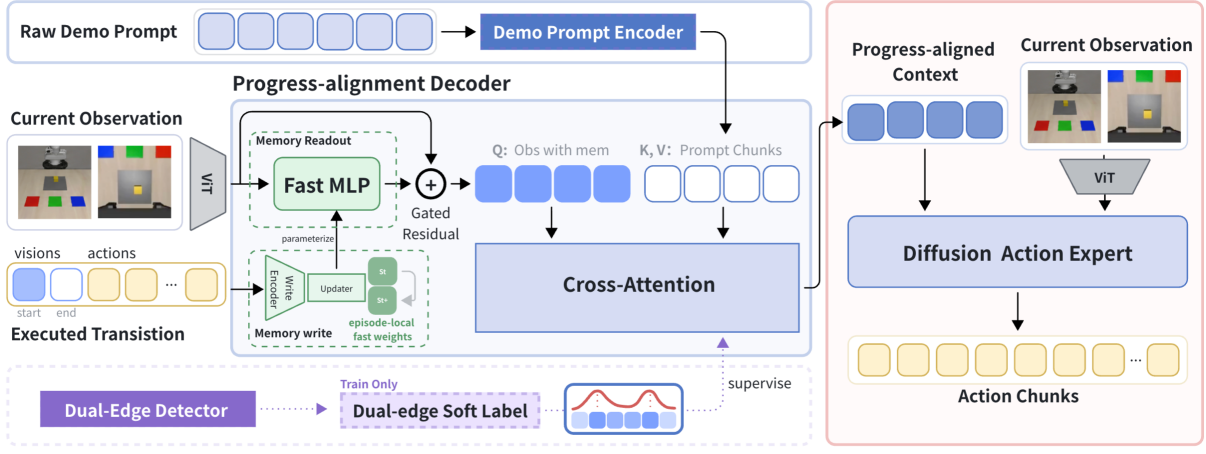


Fig. 2. Architecture overview of PACE. The demonstration prompt encoder produces ordered prompt chunks. During execution, episode-local fast weights encode realized action–observation transitions and modulate observation queries before prompt cross-attention. Training-only dual-edge attention supervision is applied to the demonstration attention. The resulting progress-aligned context and current observation condition a diffusion action expert.

action–observation transition: action a_k^e is executed under observation o_k^e , producing the subsequent observation o_{k+1}^e . At execution timestep t , o_t^e denotes the current execution observation, and $\mathcal{H}_t$ contains all realized transitions prior to action prediction at timestep t . The policy π_θ , parameterized by θ , predicts the execution action sequence $a_{t:t+H-1}^e$ over a horizon of H steps. The superscripts p and e distinguish prompt demonstration and execution quantities, respectively. The parameters θ remain fixed at test time; any episode-local state is reset between episodes. The demonstration $\mathcal{P}$ is the sole behavior specification, with no test-time stage label or progress signal.

The demonstration and execution follow the same high-level operation order but may differ in trajectory length and stage duration. Their latent progress correspondence is represented by

$$\phi : \{1, \dots, T_e\} \rightarrow \{1, \dots, T_p\}, \quad (2)$$

where T_e is the execution horizon and $\phi(t)$ maps execution timestep t to its corresponding latent position in the demonstration. The mapping ϕ is monotonic but need not advance linearly or at a constant rate. It is latent rather than explicitly predicted, and the relevant guidance may span a contiguous demonstration interval around $\phi(t)$. Given training tasks $\mathcal{T}_{\text{train}}$, we learn π_θ to maximize complete-task success on evaluation tasks $\mathcal{T}_{\text{test}}$, including unseen task compositions or held-out demonstrations, without manually annotated stage boundaries or predefined task decompositions.

IV. METHOD

A. Architecture

PACE is a demonstration-driven closed-loop method for long-horizon, multi-stage manipulation. Given a complete behavior demonstration and the current observation, PACE continually updates an episode-local state from strictly causal execution history. This state modulates cross-attention between the current observation and the demonstration, dynam-

ically producing a progress-aligned demonstration context that conditions a unified diffusion action expert.

As shown in Fig. 2, PACE comprises four components: a demonstration prompt encoder, dual-edge attention supervision, a stateful progress-alignment decoder, and a unified diffusion action expert. The prompt encoder represents the ordered demonstration structure, while dual-edge attention supervision shapes this representation during training. The decoder and action expert infer execution progress from realized history and generate closed-loop actions.

B. Demonstration Prompt Encoder

A long-horizon demonstration typically contains several behavior stages with a clear temporal order. Encoding every frame is computationally expensive and can overemphasize local visual similarity. Drawing on the temporal chunking strategy of BPP [4], we divide the demonstration into fixed-length chunks of $K = 20$ control steps, corresponding to approximately one second at 20 Hz. Actions in an incomplete final chunk are zero-padded and masked. We retain only the first visual frame of each chunk, while preserving its proprioceptive and action sequences, as summarized in Fig. 3(a).

For chunk i , a shared pretrained CLIP ViT [30], [31] encodes the first frame from each camera, with its CLS token used as the visual representation. End-effector pose and gripper state are linearly projected to the common dimension d , while the K actions are flattened and encoded by an action MLP. Let $\mathcal{V}_i = \{\mathbf{v}_i^c\}_{c=1}^C$ and $\mathcal{P}_i = \{\mathbf{p}_{i,j}\}_{j=0}^{K-1}$ denote the visual and proprioceptive features. These modalities are then fused using attention pooling [32], which uses a single learnable latent token as the query and the d -dimensional visual and proprioceptive tokens, together with the action token $\mathbf{a}_i$, as keys and values. Finally, a learnable temporal positional embedding $\mathbf{e}_i$ is added to obtain the chunk token:

$$\mathbf{z}_i = \text{AttnPool}(\mathcal{V}_i, \mathcal{P}_i, \mathbf{a}_i) + \mathbf{e}_i. \quad (3)$$

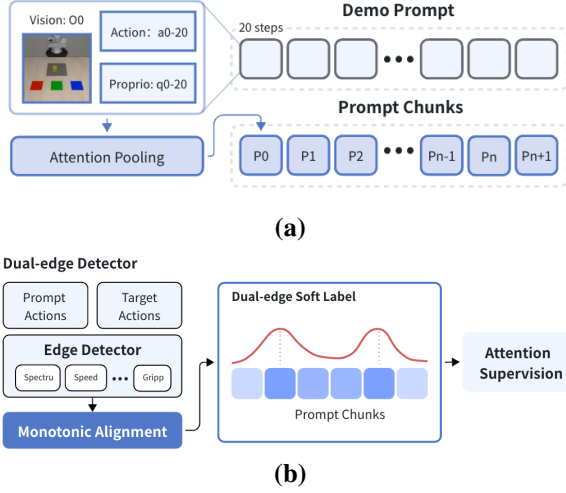


Fig. 3. Stage-structure representation in PACE. (a) Each 20-step multimodal demonstration chunk is aggregated by attention pooling into one ordered prompt token. (b) Detected prompt and target boundaries are monotonically aligned to construct a soft dual-edge target over prompt chunks for attention supervision.

Here, $i \in \{0, \dots, N_p - 1\}$ indexes demonstration chunks, $N_p = \lceil T_p/K \rceil$ is the number of prompt chunks, and C is the number of camera views. The vector $\mathbf{v}_i^c$ denotes the visual feature from camera c for chunk i , while $\mathbf{p}_{i,j}$ denotes the proprioceptive feature at within-chunk step $j \in \{0, \dots, K-1\}$. The action feature $\mathbf{a}_i$ is obtained by flattening and encoding the K actions in chunk i , and $\mathbf{e}_i \in \mathbb{R}^d$ is a learnable temporal positional embedding. $\text{AttnPool}(\cdot)$ aggregates the multimodal chunk features into a single d -dimensional token $\mathbf{z}_i$. The resulting demonstration memory $\mathbf{Z}^p = [\mathbf{z}_0, \dots, \mathbf{z}_{N_p-1}]$ is a continuous, ordered sequence at a fixed temporal resolution rather than a sequence of discrete event labels.

C. Dual-Edge Attention Supervision

PACE automatically detects stage transitions from both the target execution and prompt demonstration trajectories for dual-edge attention supervision during training. At each timestep, the supervision pipeline compares action patterns in two neighboring windows and combines changes in local action spectrum, motion direction, mean, end-effector speed, pausing behavior, jerk, and gripper state. Continuous motion channels are robustly normalized, while gripper transitions are handled separately as explicit boundary cues. Weighted change signals are filtered using peak height, prominence, and minimum-distance constraints, and the trajectory start is added as a fixed anchor.

Execution and demonstration trajectories may differ in speed, stage duration, and the number of detected boundaries. We therefore use dynamic programming to obtain a monotonic correspondence between their ordered anchor sequences, following the ordering principle of classical sequence alignment [25]. The matching cost considers relative temporal position, boundary type, repeated matches, and skipped prompt anchors. This preserves behavior order

while tolerating a small number of missing or redundant detections. Figure 3(b) summarizes the resulting training-only supervision pipeline.

For a training timestep, the correspondence identifies the left and right boundaries, μ_L and μ_R , of the matching demonstration stage. We convert them into masked Gaussian distributions over prompt chunks with $\sigma = 0.75$ and supervise both the aggregated decoder attention and each observation query:

$$\mathbf{y}_t = \frac{1}{2}\mathcal{G}_\sigma(\mu_L) + \frac{1}{2}\mathcal{G}_\sigma(\mu_R),$$

$$\mathcal{L}_{\text{edge}} = \text{CE}(\mathbf{y}_t, \bar{\alpha}_t) + \gamma \mathbb{E}_q [\text{CE}(\mathbf{y}_t, \alpha_{t,q})]. \quad (4)$$

Here, μ_L and μ_R denote the prompt-chunk indices of the aligned left and right stage boundaries for training timestep t . $\mathcal{G}_\sigma(\mu)$ is a Gaussian distribution centered at boundary location μ with standard deviation $\sigma = 0.75$, evaluated over valid prompt-chunk indices and renormalized after masking. The target distribution $\mathbf{y}_t$ is the equally weighted mixture of the two boundary-centered distributions. $\bar{\alpha}_t$ is the demonstration attention averaged across attention heads and observation queries, while $\alpha_{t,q}$ is the demonstration-attention distribution produced by observation query q . $\text{CE}(\cdot, \cdot)$ denotes cross-entropy, $\mathbb{E}_q[\cdot]$ averages over all observation queries, and γ controls the relative weight of query-level attention supervision. The dual-edge target describes a stage interval rather than a single matching frame, allowing flexible aggregation within the stage and accommodating demonstration-execution speed variation. We omit this supervision for an open final stage without a reliable right boundary.

D. Stateful Progress-Alignment Decoder

Stage structure alone does not identify the robot’s current progress when visually similar states recur across stages. PACE therefore maintains an episode-local fast-weight memory, represented by a fixed $d_m \rightarrow h_m \rightarrow d_m$ two-layer MLP f_s . Its state $\mathbf{s}_k = \{W_1, b_1, W_2, b_2\}$ contains all MLP weights and biases rather than a low-rank subset. Thus, “fixed-size” means that the parameterized state has constant size and stores no growing trajectory buffer. Following the TTT-MLP view of a learnable model as recurrent state [18], [19], PACE updates this state after every completed execution segment of $S = 12$ control steps, independently of the $K = 20$ -step prompt chunks. Each completed segment represents realized action-observation transitions and is encoded from its valid actions and start/end multi-camera observation features; the resulting representation is projected into a normalized key $\mathbf{k}_k$ and value $\mathbf{v}_k^{\text{mem}}$.

Each episode starts from a shared learnable initialization $\mathbf{s}_0$, and each valid write performs a local key-value reconstruction update:

$$\mathcal{L}_{\text{write}}^k = \frac{1}{d_m} \|\mathbf{f}_{\mathbf{s}_k}(\mathbf{k}_k) - \mathbf{v}_k^{\text{mem}}\|_2^2,$$

$$\mathbf{s}_{k+1} = \mathbf{s}_k - \eta \nabla_{\mathbf{s}_k} \mathcal{L}_{\text{write}}^k. \quad (5)$$

Here, k indexes completed segments, d_m is the memory dimension, and η is a learnable inner-update step size. The reconstruction objective defines only the state transition and

is not added to the outer loss; the shared initialization, projections, and η are learned by the outer control objectives. Training backpropagates through at most 16 consecutive writes and applies stop-gradient between truncated-BPTT windows. Writes are strictly causal: only segments preceding the prediction time are used in training, and only realized action–observation transitions are written during deployment. With $d_m = 64$ and $h_m = 128$, the state contains 16,576 scalars and each core fast-MLP write requires approximately $5d_m h_m \approx 4.1 \times 10^4$ multiply–accumulate operations, excluding segment encoding; this cost does not grow with episode length.

Each decoder layer applies observation self-attention, injects a gated fast-memory readout, and only then cross-attends to the complete demonstration:

$$\begin{aligned} \Delta \mathbf{Q}_t^l &= W_o^l f_{s_{k(t)}} \left(W_r^l \text{LN}(\tilde{\mathbf{Q}}_t^l) \right), \\ \hat{\mathbf{Q}}_t^l &= \tilde{\mathbf{Q}}_t^l + \tanh(g_l) \Delta \mathbf{Q}_t^l, \quad \mathbf{C}_t^l = \text{MHA}(\hat{\mathbf{Q}}_t^l, \mathbf{Z}^p, \mathbf{Z}^p). \end{aligned} \quad (6)$$

Here, LN denotes layer normalization, and MHA denotes multi-head attention. $k(t)$ denotes the most recently completed execution segment at timestep t , $\tilde{\mathbf{Q}}_t^l$ is the observation representation after self-attention, W_r^l and W_o^l project into and out of the memory space, and g_l gates the history-dependent correction. The modulated observation tokens $\hat{\mathbf{Q}}_t^l$ query the complete demonstration memory $\mathbf{Z}^p$, which supplies keys and values. Because memory modulation precedes prompt cross-attention, the same observation can retrieve different demonstration stages under different execution histories. The final context $\mathbf{C}_t = \mathbf{C}_t^L$ is concatenated with the original observation tokens to condition action generation.

The progress-alignment decoder component in Figure 2 summarizes the causal memory write, fast-weight readout, and history-modulated prompt cross-attention.

E. Unified Diffusion Action Expert

PACE uses a single conditional 1D U-Net across tasks and behavior stages rather than maintaining stage-specific policies. Task semantics and current progress are specified implicitly by the current observation and progress-aligned demonstration context. We train the expert with the standard diffusion denoising objective [33], [34] and jointly apply the dual-edge attention supervision loss to its final-layer demonstration attention. Entropy-based effective-support regularization further prevents attention from spreading uniformly across the complete demonstration while preserving flexible weighting within the relevant interval. The complete objective is

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_{\text{edge}}(u) \mathcal{L}_{\text{edge}} + \lambda_{\text{sup}} \mathcal{L}_{\text{sup}}. \quad (7)$$

Here, $\mathcal{L}_{\text{diff}}$ is the standard diffusion denoising loss, $\mathcal{L}_{\text{edge}}$ is the dual-edge attention supervision loss in Eq. (4), and $\mathcal{L}_{\text{sup}}$ is the effective-support regularization term on demonstration attention. The scalar u denotes training progress, $\lambda_{\text{edge}}(u)$ is a schedule-dependent weight that is gradually increased during training, and λ_{sup} is the fixed weight of the effective-support regularization. Thus, the action objective remains

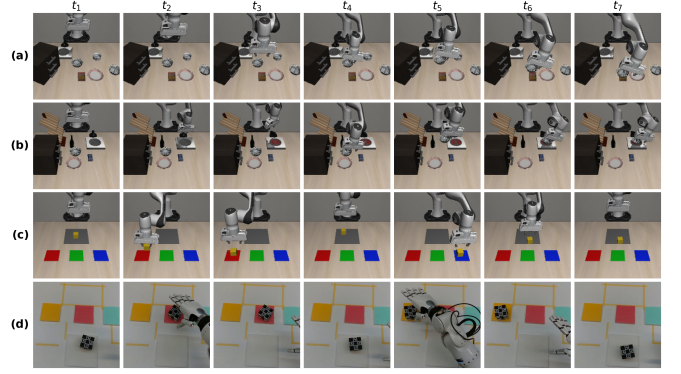


Fig. 4. Representative execution sequences: (a) simulation Spatial Combination, (b) simulation Goal Chain, (c) simulation Block Routing, and (d) real-world Block Routing.

shared across all stages while the auxiliary terms shape how the demonstration is read.

At inference time, dual-edge attention supervision is removed entirely; no explicit event segmentation or discrete stage prediction is required. PACE updates its fast state from the current observation and realized execution history, constructs a continuous progress-aligned context, executes part of the action sequence produced by the diffusion expert, and replans after incorporating the new outcome. This yields closed-loop progress alignment and action generation.

V. EXPERIMENTS

This section evaluates the overall closed-loop performance of PACE on long-horizon, multi-stage demonstration-conditioned manipulation tasks, using complete-task success as the primary criterion. We defer analysis of stage confusion, demonstration structure, and the progress-alignment mechanism to Sec. VI.

A. Experimental Setup

Simulation tasks. We evaluate three simulation task families with different stage compositions and execution structures, illustrated in Fig. 4. *LIBERO-Gen Goal Chain* concatenates a prerequisite interaction, such as opening a drawer, with a subsequent object-centric goal, testing unseen compositions of goals and behavior chains. *LIBERO-Gen Spatial Combination* recombines manipulated objects and target locations from LIBERO-Spatial [35] to evaluate demonstration-conditioned generalization to unseen spatial relations; both LIBERO-Gen suites follow BPP [4]. *Block Routing* repeatedly moves the same block among red, green, and blue target regions, returning it to a common home region between consecutive operations. It contains three single-region routes and nine ordered two-region routes, for 12 tasks in total, and therefore presents recurring observations across behavior stages. Together, these tasks cover the two main challenges considered by PACE: cross-stage observation aliasing and demonstration–execution temporal-scale mismatch.

Real-robot tasks. We further evaluate PACE on real-robot Block Routing under visual perturbations, action errors, and stochastic interaction outcomes, as illustrated in Fig. 4. The task mirrors the simulation setup and allows us to examine behavioral consistency across simulation and reality.

Baselines. Behavior Prompting Policy (BPP) [4] is our principal baseline and uses the same form of complete behavior demonstration and current observation as PACE. We additionally compare with $\pi_{0.5}$ [36] in simulation. BPP and PACE use the same training split and checkpoint-selection protocol. All methods are evaluated with the same test demonstrations, success criteria, and rollout protocol.

Evaluation metric. Our primary metric is episode-level task success rate. An episode is successful only if every operation stage is completed in the correct order. We evaluate each task with multiple independent demonstrations and rollout seeds and report the mean over all test episodes. No stage label, task-progress signal, or semantic subgoal is provided at test time.

B. Simulation Results

Figure 5(a)–(b) summarizes complete-task success rates on the three simulation task families.

On the unseen compositions in Fig. 5(a), PACE achieves 83.3% on Spatial Combination and 94.0% on Goal Chain, improving over BPP by 4.2 and 5.1 percentage points and over $\pi_{0.5}$ by 3.4 and 14.8 points, respectively. On the seen Block Routing tasks in Fig. 5(b), PACE reaches 73.3% on 2-step tasks and 37.8% on 4-step tasks. These results exceed BPP by 40.0 and 34.5 points and $\pi_{0.5}$ by 60.0 and 37.6 points, respectively. The larger gains on Block Routing, particularly when BPP and $\pi_{0.5}$ fall to 3.3% and 0.2% on 4-step tasks, show that PACE better preserves complete-task performance as stage dependencies and recurring states increase.

PACE receives neither test-time stage labels nor stage-specific action experts. Its gains therefore arise from progress-dependent interpretation of the same demonstration condition rather than additional task-semantic supervision.

C. Real-Robot Results

Real-world execution exhibits greater variation in action duration, contact error, and visual appearance than simulation, so the time required to complete a behavior stage often differs from the demonstration. Figure 5(c) reports the corresponding evaluation.

PACE achieves 90.0% success on the real-robot 2-step tasks, compared with 40.0% for BPP, an improvement of 50.0 percentage points. On the more demanding 4-step tasks, PACE reaches 43.3%, compared with 3.44% for BPP, improving success by 39.9 percentage points.

VI. ABLATIONS

This section addresses three questions: (1) whether PACE’s gains primarily arise from reducing stage confusion; (2) whether demonstration stage structure and stateful progress alignment are both required; and (3) whether the

learned demonstration attention dynamically tracks execution progress within the demonstration.

A. Does PACE Reduce Stage Confusion?

We analyze this issue on the two-step Block Routing task, where the robot will repeatedly pass through the same region, creating distinct stages with highly similar observations that can lead to stage confusion. Here, we compare BPP, PACE, and two ablated variants of PACE. For each method, we conduct 10 rollouts on each task, resulting in 30 evaluation rollouts per method. Figure 6 reports the success rate and failure-mode composition of each method.

We categorize failures into three types: *wrong-stage*, where the policy executes behavior associated with an incorrect stage of the demonstrated sequence; *wrong-color*, where the policy proceeds with the correct stage but selects an incorrect target color; and *wrong-grasp*, where the intended stage and target are correct but execution fails due to low-level grasping or placement errors.

In BPP, wrong-stage failures occur 14 times, accounting for 46.7% of all trials, whereas PACE produces only one such failure, accounting for 3.3%. Among failed rollouts, the proportion of wrong-stage failures decreases from 56.0% in BPP to 12.5% in PACE. In both the Effective-support regularization only and Dual-edge attention supervision only variants, wrong-stage failures remain the dominant failure mode as in BPP. Together with the 56.6-percentage-point success-rate improvement from BPP to PACE in Fig. 6, these results support the hypothesis that PACE improves long-horizon manipulation primarily by suppressing stage confusion, with most of this gain attributable to the fast-weight memory (FWM).

B. Role of Stage Structure and Stateful Progress Alignment

Figure 6 separates the stage-structure losses (effective-support regularization and dual-edge attention supervision) from the episodic fast-weight state (FWM). Neither structural loss improves success in isolation, while the full model substantially outperforms BPP. This indicates that stage supervision alone cannot resolve recurring observations; it must interact with an execution state that changes demonstration retrieval as progress evolves.

C. Does Demonstration Attention Track Execution Progress?

Figure 7 compares BPP, PACE without fast-weight memory (FWM), and full PACE on the same two-step Block Routing task, prompt demonstration, training seed, and rollout seed. BPP and PACE without FWM produce representative failures, whereas full PACE succeeds. BPP attention remains diffuse and nearly stationary. Without FWM, the dual-edge structure emerges, but attention repeatedly switches between stages under visually aliased observations. In contrast, full PACE maintains an approximately monotonic attention trajectory aligned with execution progress, suggesting that FWM helps prevent retrieval of visually compatible but stage-inappropriate demonstration regions.

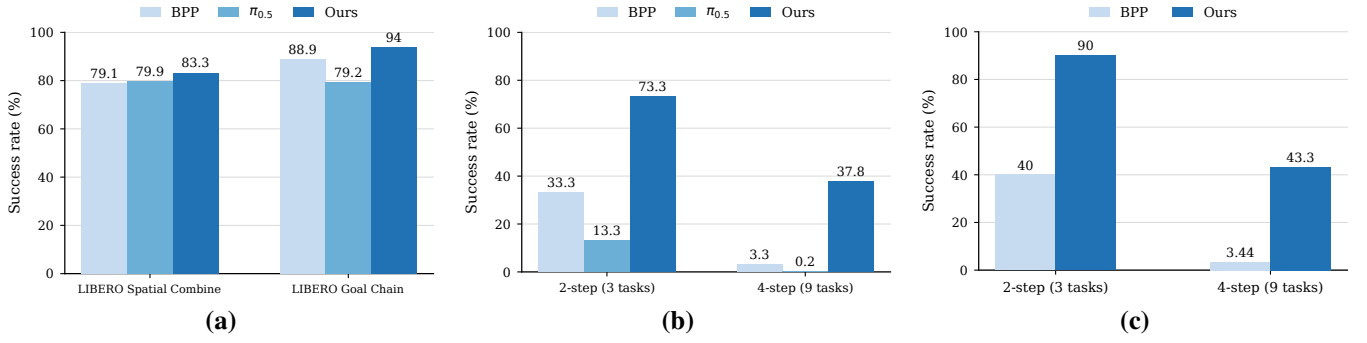


Fig. 5. Complete-task success rates. The horizontal axis denotes the task category, and the vertical axis reports the mean complete-task success rate. (a) **Simulation Experiment 1:** LIBERO-Gen Spatial Combination and LIBERO-Gen Goal Chain each contain 10 tasks whose stage compositions are unseen during training. Each task is evaluated with 10 rollouts, and the success rate is averaged across tasks. (b) **Simulation Experiment 2:** Block Routing consists of 2-step and 4-step tasks, with 10 training demonstrations for each task. A 2-step task moves the block to one of the red, green, or blue target regions and then returns it to the home region, yielding three tasks. A 4-step task concatenates an ordered pair of 2-step tasks—for example, Home $\rightarrow$ Red $\rightarrow$ Home $\rightarrow$ Green $\rightarrow$ Home—yielding $3 \times 3 = 9$ tasks. Each task is evaluated with 10 rollouts, and the mean success rate is reported for each category. (c) **Real-robot Block Routing:** The task follows the same setup as (b), using 15 training demonstrations and 10 evaluation rollouts per task; the mean success rate is reported for each category.

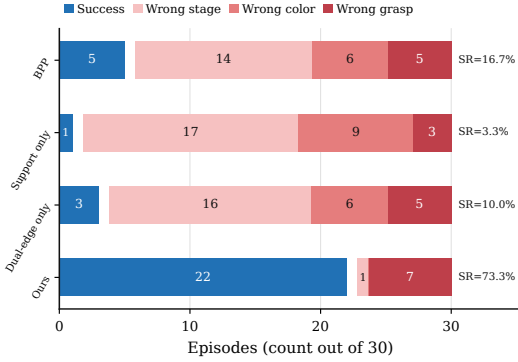


Fig. 6. Complete outcome composition on the two-step Block Routing task over 30 rollouts. Each horizontal bar partitions 30 episodes into successes, wrong-stage failures, wrong-color failures, and wrong-grasp failures. Failure subsegments are uniformly compressed to retain the common 30-episode endpoint, while labels report exact counts. The corresponding success rate is reported at right.

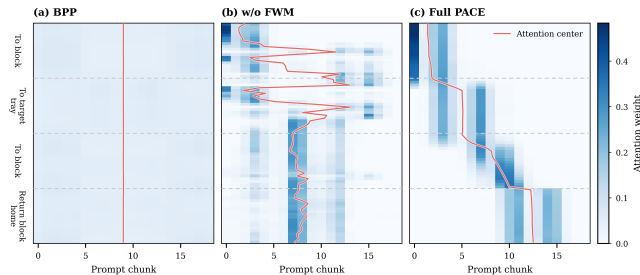


Fig. 7. Attention maps for two-step Block Routing: (a) BPP and (b) PACE w/o FWM show representative failures, while (c) Full PACE shows a representative success. Rows are aligned to four intended stages—*To block*, *To target tray*, *To block*, and *Return block home*. Blue intensity represents normalized attention weight, gray dashed lines separate intervals, and red curves mark the attention center of mass.

To isolate the effect of execution history, we hold the aliased current observation X^* and demonstration P fixed while varying only the executed episode prefix $\mathcal{H}^{(r)}$:

$$\mathbf{s}^{(r)} = \text{Update}(\mathbf{s}_0, \mathcal{H}^{(r)}), \quad \alpha^{(r)} = \alpha(P \mid X^*, \mathbf{s}^{(r)}). \quad (8)$$

Here, Update applies the causal memory writes in Eq. (5), and $\alpha^{(r)}$ is the resulting demonstration-attention distribution. We compare matched, earlier-stage, later-stage, shuffled, and reset histories.

Figure 8 provides qualitative evidence that demonstration retrieval depends not only on the current observation but also on the causal execution history. In the upper panel, we fix the current observation from the first *To block* stage, which is visually similar to observations from the second *To block* stage. Replacing the corresponding fast-state history with the execution history of the second occurrence shifts the attention center from the early prompt chunks to the later matched stage, whereas resetting the fast state moves it back toward the earlier stage. The lower panel, based on a randomly selected current stage, exhibits the same history-dependent behavior: the matched history concentrates attention within the corresponding stage boundaries, while earlier- and later-stage histories shift the attention center toward their respective demonstration regions. Although the shuffled history retains a similar attention center in this example, its overall attention distribution changes. These results indicate that the episode-local fast-weight state makes demonstration retrieval progress-dependent, allowing the same current observation to attend to different demonstration stages under different realized execution histories.

VII. CONCLUSION AND LIMITATIONS

We presented PACE, a demonstration-driven closed-loop method for long-horizon, multi-stage manipulation. PACE represents a complete demonstration as an ordered multi-modal prompt, learns its stage structure through training-only dual-edge attention supervision, and uses a causal episode-local fast-weight memory to continually align demonstration

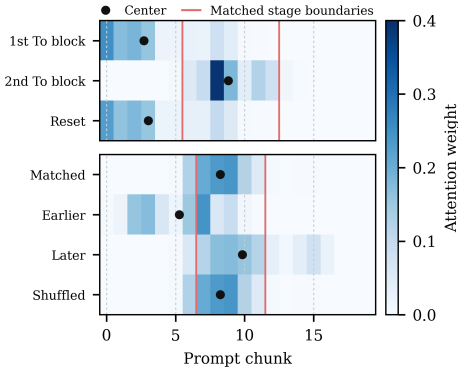


Fig. 8. Counterfactual history conditioning with the current observation and demonstration held fixed. The upper panel directly contrasts the visually aliased first and second *To block* occurrences. The lower panel compares matched, earlier, later, and shuffled histories for a randomly selected sample. Black markers denote the attention center of mass, and red lines denote the matched-stage boundaries.

attention with realized execution progress. A unified diffusion action expert then generates actions from the aligned context without explicit stage labels or stage-specific policies. Results on compositional simulation tasks and Block Routing show improved complete-task success and a substantial reduction in wrong-stage failures, supporting the joint role of structured demonstration supervision and stateful progress alignment.

Limitations. The current evaluation covers a limited set of task families and relatively small rollout counts, so broader validation across embodiments, environments, and longer behavior chains is needed. Dual-edge attention supervision relies on action-change cues and assumes a monotonic correspondence between execution and demonstration stages, which may be restrictive for tasks with optional, repeated, or reordered subgoals. Fixed-duration prompt chunks can also blur transitions shorter than the chosen temporal resolution, while fast-weight adaptation introduces additional state-management and computational overhead. Future work should learn boundary discovery end to end, support non-monotonic task structures, and evaluate adaptive temporal resolutions and more efficient episodic memory updates.

REFERENCES

- [1] Octo Model Team, D. Ghosh, H. Walke *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [2] V. Jain, M. Attarian, N. J. Joshi *et al.*, “Vid2Robot: End-to-end video-conditioned policy learning with cross-attention transformers,” *arXiv preprint arXiv:2403.12943*, 2024.
- [3] Y. Jiang, A. Gupta, Z. Zhang, G. Wang, Y. Dou, Y. Chen, L. Fei-Fei, A. Anandkumar, Y. Zhu, and L. Fan, “VIMA: Robot manipulation with multimodal prompts,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 202, 2023, pp. 14975–15022.
- [4] A. Patel, B. Pekarek, J. E. C. Hernandez, and S. Song, “Behavior prompting policy: Demonstrations as prompts for manipulation,” *arXiv preprint arXiv:2606.30457*, 2026.
- [5] Y. Duan, M. Andrychowicz, B. Stadie, J. Ho, J. Schneider, I. Sutskever, P. Abbeel, and W. Zaremba, “One-shot imitation learning,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [6] S. James, M. Bloesch, and A. J. Davison, “Task-embedded control networks for few-shot imitation learning,” in *Proc. Conf. Robot Learn. (CoRL)*, vol. 87, 2018, pp. 783–795.
- [7] C. Devin, D. Geng, P. Abbeel, T. Darrell, and S. Levine, “Compositional plan vectors,” in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [8] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song, “XSkill: Cross-embodiment skill discovery,” in *Proc. Conf. Robot Learn. (CoRL)*, 2023.
- [9] L. Fu, H. Huang, G. Datta, L. Y. Chen, W. C.-H. Panitch, F. Liu, H. Li, and K. Goldberg, “In-context imitation learning via next-token prediction,” *arXiv preprint arXiv:2408.15980*, 2024.
- [10] R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning,” *Artif. Intell.*, vol. 112, no. 1–2, pp. 181–211, 1999.
- [11] K. Pertsch, Y. Lee, and J. J. Lim, “Accelerating reinforcement learning with learned skill priors,” in *Proc. Conf. Robot Learn. (CoRL)*, vol. 155, 2021, pp. 188–204.
- [12] K. Shiarlis, M. Wulfmeier, S. Salter, S. Whiteson, and I. Posner, “TACO: Learning task decomposition via temporal alignment for control,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 80, 2018, pp. 4654–4663.
- [13] T. Kipf, Y. Li, H. Dai, V. Zambaldi, A. Sanchez-Gonzalez, E. Grefenstette, P. Kohli, and P. Battaglia, “CompILE: Compositional learning and execution,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 97, 2019, pp. 3418–3428.
- [14] P. Sermanet, C. Lynch, Y. Chebotar, J. Hsu, E. Jang, S. Schaal, and S. Levine, “Time-contrastive networks: Self-supervised learning from video,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 1134–1141.
- [15] D. Dwibedi, Y. Aytar, J. Tompson, P. Sermanet, and A. Zisserman, “Temporal cycle-consistency learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 1801–1810.
- [16] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, and R. Salakhutdinov, “Transformer-XL: Attentive language models beyond a fixed-length context,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2019, pp. 2978–2988.
- [17] J. Ba, G. E. Hinton, V. Mnih, J. Z. Leibo, and C. Ionescu, “Using fast weights to attend to the recent past,” in *Adv. Neural Inf. Process. Syst.*, vol. 29, 2016.
- [18] Y. Sun, X. Li, K. Dalal, J. Xu, A. Vikram, G. Zhang, Y. Dubois, X. Chen, X. Wang, S. Koyejo, T. Hashimoto, and C. Guestrin, “Learning to (learn at test time): RNNs with expressive hidden states,” *arXiv preprint arXiv:2407.04620*, 2024.
- [19] Y. Jiang, Y. Chebotar, R. Zheng, F. Hu, Y. Ge, J. Wu, T. Dai, S. Reed, L. Fei-Fei, Y. Zhu, and L. J. Fan, “RoboTTT: Context scaling for robot policies,” *arXiv preprint arXiv:2607.15275*, 2026.
- [20] J. M. Zacks and B. Tversky, “Event structure in perception and conception,” *Psychol. Bull.*, vol. 127, no. 1, pp. 3–21, 2001.
- [21] J. M. Zacks, N. K. Speer, K. M. Swallow, T. S. Braver, and J. R. Reynolds, “Event perception: A mind-brain perspective,” *Psychol. Bull.*, vol. 133, no. 2, pp. 273–293, 2007.
- [22] C. A. Kurby and J. M. Zacks, “Segmentation in the perception and memory of events,” *Trends Cogn. Sci.*, vol. 12, no. 2, pp. 72–79, 2008.
- [23] G. Papagiannis, N. D. Palo, P. Vitiello, and E. Johns, “R+X: Retrieval and execution from everyday human videos,” *arXiv preprint arXiv:2407.12957*, 2024.
- [24] K. Kedia, P. Dan, A. Chao, M. A. Pace, and S. Choudhury, “One-shot imitation under mismatched execution,” *arXiv preprint arXiv:2409.06615*, 2024.
- [25] H. Sakoe and S. Chiba, “Dynamic programming algorithm optimization for spoken word recognition,” *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 26, no. 1, pp. 43–49, 1978.
- [26] M. Cuturi, “Sinkhorn distances: Lightspeed computation of optimal transport,” in *Adv. Neural Inf. Process. Syst.*, vol. 26, 2013.
- [27] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, “Diversity is all you need: Learning skills without a reward function,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [28] C. Lynch, M. Khansari, T. Xiao, V. Kumar, J. Tompson, S. Levine, and P. Sermanet, “Learning latent plans from play,” in *Proc. Conf. Robot Learn. (CoRL)*, vol. 100, 2020, pp. 1113–1132.
- [29] C. Wang, L. Fan, J. Sun, R. Zhang, L. Fei-Fei, D. Xu, Y. Zhu, and A. Anandkumar, “MimicPlay: Long-horizon imitation learning by watching human play,” in *Proc. Conf. Robot Learn. (CoRL)*, vol. 229, 2023, pp. 201–221.
- [30] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 8748–8763.
- [31] A. Dosovitskiy, L. Beyer, A. Kolesnikov *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [33] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6840–6851.
- [34] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.
- [35] B. Liu, Y. Zhu, C. Gao, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *Adv. Neural Inf. Process. Syst. Datasets Benchmarks Track*, vol. 36, 2023.
- [36] K. Black, N. Brown, J. Darpinian *et al.*, " $\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.